

第五章

1. (1) A (2) D (3) D (4) C (5) A (6) C (7) C (8) B
(9) D (10) C (11) B (12) C (13) A (14) C (15) A

2. (1) 先序, 中序, 后序的访问顺序分别为

根左右, 左根右, 左右根

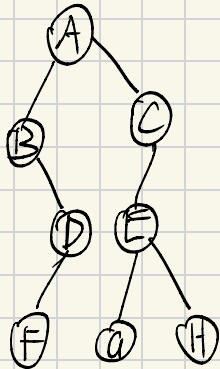
由条件 ① ② ③ 可知

三种顺序的访问序列相同

又 ④ 可知

该二叉树只能有 1 个结点

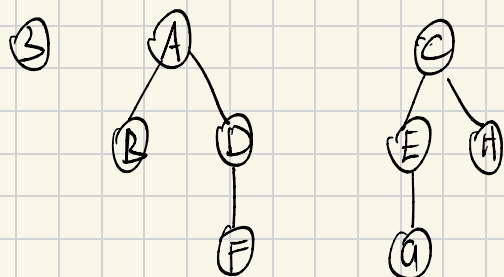
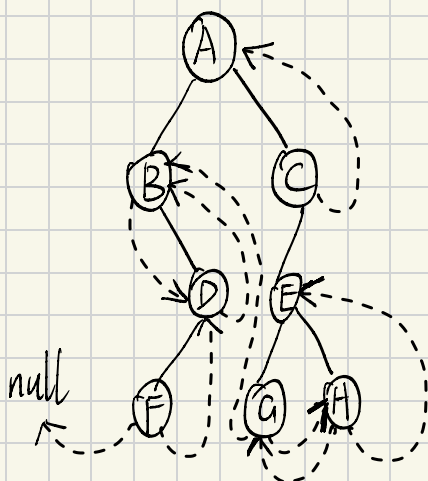
(2) ① 由先序和中序序列可得



② 后序序列为

F D B G H E C A

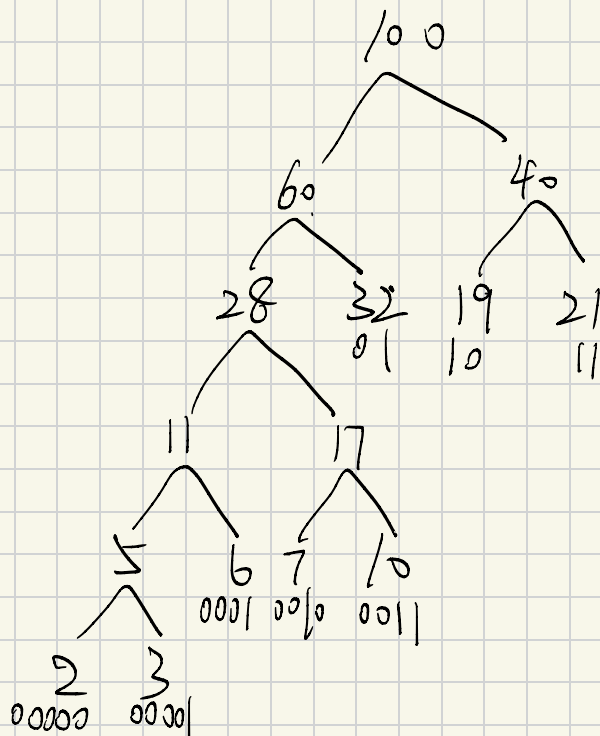
后序线索树为



(3) ① 不妨将频率均乘100方便计算

分别为 7、19、2、6、32、3、4、10

构造 Huffman 树



采取左0右1

0.07 : 00/0

0.19 : 10

0.02 : 00000

0.06 : 0001

0.32 : 01

0.03 : 00001

0.21 : 11

0.10 : 0011

② 共8个字母则需要3位二进制位

0.07: 000

0.19: 001

0.02: 010

0.06: 011

0.32: 100

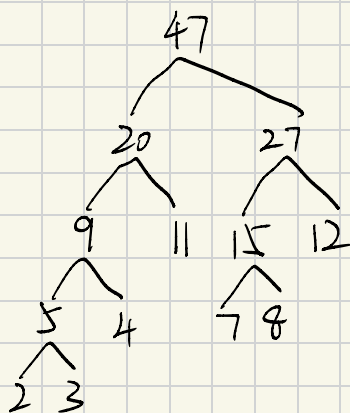
0.03: 101

0.21: 110

0.10: 111

③ 哈夫曼编码的优点在于可以根据不同字母的频率来设计编码, 因此可以最大限度地减少相同文本编码的长度。然而正因, 长短不一的编码造成解码的效率相对低下。相反地, 等长的二进制编码则编码长度相对较长, 但解码效率相对较高

(4) 哈夫曼树如右。



初态

i	weight	parent	lchild	rchild
1	3	0	0	0
2	12	0	0	0
3	7	0	0	0
4	4	0	0	0
5	2	0	0	0
6	8	0	0	0
7	11	0	0	0
8		0	0	0
9		0	0	0
10		0	0	0
11		0	0	0
12		0	0	0
13		0	0	0

终态:

i	weight	parent	lchild	rchild
1	3	8	0	0
2	12	12	0	0
3	7	10	0	0
4	4	9	0	0
5	2	8	0	0
6	8	10	0	0
7	11	11	0	0
8	5	9	5	1
9	9	11	8	4
10	15	12	6	3
11	20	13	9	7
12	27	13	10	2
13	47	0	12	11

```

3. c) int leaf_node_count(BiTree T) {
    if (T == NULL) {
        return 0;
    } else if (T->lchild == NULL && T->rchild == NULL) {
        return 1;
    } else {
        return leaf_node_count(T->lchild) +
               leaf_node_count(T->rchild);
    }
}

```

```

(2) status is_equal(BiTree T1, BiTree T2) {
    bool T1_isnull = (T1 == NULL);
    bool T2_isnull = (T2 == NULL);
    if (T1_isnull != T2_isnull) {
        return ERROR;
    } else if (T1_isnull && T2_isnull) {
        return OK;
    } else {

```

```
if (T1→data != T2→data){
```

```
    return ERROR;
```

```
} else {
```

```
    return is_equal(T1→lchild,
```

```
    T2→lchild) && is_equal
```

```
    (T1→rchild, T2→rchild);
```

```
}
```

```
}
```

```
}
```

```
(3) void lr_change(BiTree &T){
```

```
    BiTree temp;
```

```
    if (T→lchild == NULL && T→rchild == NULL){
```

```
        return;
```

```
    } else {
```

```
        temp = T→lchild;
```

```
        T→lchild = T→rchild;
```

```
        T→rchild = temp;
```

```
}
```

```
l r - change ( T  $\rightarrow$  lchild );  
l r - change ( T  $\rightarrow$  rchild );
```

y

```
(6) int leaf_node_level ( Bitree T ) {
```

```
    int num = 0;
```

```
    if ( T ) {
```

```
        QueueInit ( Q );
```

```
        QueueIn ( Q, T );
```

```
        while ( ! is_empty ( Q ) ) {
```

```
            P = QueueOut ( Q );
```

```
            if ( P  $\rightarrow$  lchild && ! P  $\rightarrow$  rchild ||  
                ! P  $\rightarrow$  lchild && P  $\rightarrow$  rchild ) {
```

```
                num ++;
```

y

```
            if ( P  $\rightarrow$  lchild ) {
```

```
                QueueIn ( Q, P  $\rightarrow$  lchild );
```

y

```
            if ( P  $\rightarrow$  rchild ) {
```

QueueIn(Q, p $\Rightarrow$ rchild);

y

p

return num;

y